TRAIN SIM WORLD · VEHICLE WORKFLOW GUIDE

# Part 6

## View polish, audio, test, ship

Once the box drives: the parts that make it a locomotive. Lights and particles on the view, the audio actor, collision and LOD housekeeping, the pre-flight checklist that catches most “it does not spawn” cases, how to interrogate the running editor when it does not, and what is known about distributing the plugin.

### In this part


1. Lights
2. Particles, wheel blur and the view component
3. Collision, blockers and LOD management
4. The RVA: audio that follows the simulation
5. Pre-flight checklist
6. Interrogating the running editor
7. Troubleshooting
8. Shipping the plugin
9. What a finished vehicle contains


Component names and the Python snippets are those used in the live inspection (**verified**). Packaging and distribution were not covered by the inspection or the supplied sources beyond Dovetail's FAQ; that section is marked **check** throughout.

# 1 · Lights

A `LinearElectricLightComponent` is a tiny controller: it takes a 0..1 value and maps it through `out_intensity_range` to a real light or an emissive material parameter. The pattern on Thomas: Headlights VHID output → RVV event → `HeadlightController` → `Headlight` (SpotLight). The Class 47 has twenty of them: tail, marker, cab, desk and warning lamps. **verified**

1. RVM: PushButtonComponent `Headlights`, TOGGLE, InputIdentifier `Headlights`; add VHID presets `HeadlightsOn` / `Off` on the physics component.
2. RVV: a SpotLight at the lamp position (a `socket_Headlight` empty in Blender gives you the spot), a `LinearElectricLightComponent` `HeadlightController` pointing at it.
3. RVV event graph: bind the Headlights hitbox's `OnOutputValueChanged` → set the controller's value.
4. Lamp glass: an emissive parameter on the material instance, driven by the same controller.

# 2 · Particles, wheel blur and the view component

The inherited `RailVehicleViewComponent` holds `particle_system_definitions[]` (Arcing_Sparks, Water_Spray, Kickup, each with a Niagara system). The engine adds the three RailVehicleParticleSystem components itself and spawns them at the `Wheel_*` sockets, which is why the add-on generates those sockets at the rail contact. Also here: wheel-slip sparks, impact volume range (buffer clashes), the per-axle joint clack, rail singing (an ActorValueBinding reading a sim parameter), and `enable_coupler_vhid`. Thomas adds Niagara smoke at the chimney and whistle, and an instanced wheel-blur mesh switched by the RVM's `OnVehicleSpeedChanged`. **verified**

# 3 · Collision, blockers and LOD management

- Walkable collision is the `UCX_` static mesh component, not the skeletal mesh's physics asset; in some vehicles it doubles as the audio occluder.
- `Blocker_F/B` boxes stop the player walking off a footplate that has no door.
- `ComponentLODManagementComponent` disables lights, Niagara and the cab mesh beyond set distances; put it on every RVV.
- Skeletal mesh LODs: generate in the asset's LOD settings; the cab mesh can simply be culled by distance.

# 4 · The RVA: audio that follows the simulation

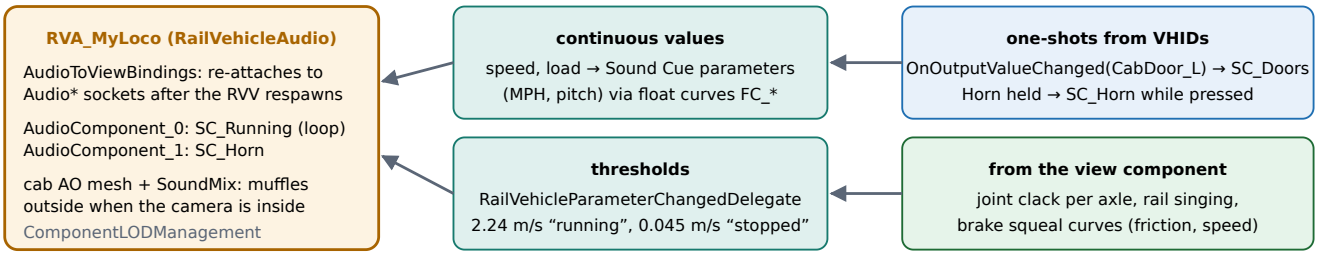



**Figure 6.1 · How Thomas's audio is wired.** 114 sound waves, 15 cues, 6 attenuations, 8 curves. A boilerplate needs two cues: a running loop and a horn. verified

1. Blueprint Class ▸ **RailVehicleAudio** ▸ `RVA_MyLoco` in `Audio/RVA`; add it to the RVD's `AdditionalViewActorAssets`.
2. Import two waves, make cues `SC_MyLoco_Running` (looping) and `SC_MyLoco_Horn`, an attenuation `A_MyLoco`.
3. Two AudioComponents attached to `AudioRunning` / `AudioWhistle` sockets; an `AudioToViewBindings` component so they re-attach when the view respawns.
4. Register a speed threshold delegate to switch the running loop on; bind the Horn VHID event to play the horn while held.
5. Later: pitch curves vs speed, cab occlusion mesh and SoundMix, door and coupler cues.

> **No RVA at all for a wagon**
>
> The streamer leaves `AdditionalViewActorAssets` empty and keeps a wagon's sounds on the RVV's view component through its value bindings: a speed binding from the Simulation component's first axle (he found the MPH parameter worked where km/h did not), a “rumble speed” binding shaped by a float curve borrowed from one of the game's mineral wagons, and an axle-speed binding, all updated every tick. The joint clack and rumble then come for free. community

# 5 · Pre-flight checklist

| ✓ | Before every test |
|---|---|
| ☐ | RVD: ModelActorAsset and ViewActorAsset set; RVM and RVV Reference Actors point at each other |
| ☐ | RVM has a component named `Simulation` with an SA assigned; the SA has the nodes the Blueprints and ABP name |
| ☐ | Every RVV VirtualHIDInteractionComponent.InteractionModelComponent matches an RVM component name exactly (case and spaces) |

| ✓ | Before every test |
|---|---|
| ☐ | Every lever's InteractionEnvironmentComponent names an environment that exists on the RVM; the camera and seat on the RVV name the same one |
| ☐ | Skeletal mesh component is named `SK_RailVehicle`; ABP parent is RailVehicleAnimInstance; Axles list filled |
| ☐ | Formation contains the RVD; RVD flags drivable, selectable, available for formation editor |
| ☐ | Everything saved and the plugin enabled before pressing Play |

# 6 · Interrogating the running editor

The editor's Python can read anything, cooked or not, while Play In Editor runs (Output Log ▸ Cmd ▸ `py`, or the Python console). These read-only snippets produced the inspection this set is based on: **verified**

```
import unreal
w = unreal.EditorLevelLibrary.get_game_world()                    # the PIE world
actors = unreal.GameplayStatics.get_all_actors_of_class(w, unreal.Actor)

# which vehicle am I in?  (walk up from the driver pawn)
p = unreal.GameplayStatics.get_player_controller(w,
0).get_controlled_pawn().get_attach_parent_actor()
while p: print(p.get_class().get_name(), p.get_name()); p = p.get_attach_parent_actor()

# list a vehicle's components with classes
rvm = [a for a in actors if a.get_name() == 'RVM_MyLoco_C_0'][0]
for c in rvm.get_components_by_class(unreal.ActorComponent): print(c.get_class().get_name(), '|',
c.get_name())

# dump every property of a component (incl. cooked) to the log; then read
Saved/Logs/TS2Prototype.log
unreal.SystemLibrary.execute_console_command(w, 'obj dump ' + c.get_path_name())

# the live SimuGraph node list and values
unreal.SystemLibrary.execute_console_command(w, 'obj dump ' + rvm.get_path_name() +
'.Simulation.SimulationRuntime_1')

# coupler id of a Dovetail vehicle, to match your own
for c in rvm.get_components_by_class(unreal.ActorComponent):
    if c.get_class().get_name() == 'CouplerComponent': print(c.get_name(),
c.get_editor_property('coupler_id'))
```

Dovetail's RVM template actors sit at (0,0,0) and the placed ones are RVV_ / RVA_; `get_editor_property` works on cooked objects for most fields and `obj dump` catches the rest.

# 7 · Troubleshooting

| Symptom | Look at |
|---|---|
| Not in the spawn list | Formation missing or RVD flags; plugin not enabled; Show Plugin Content. |

| Symptom | Look at |
|---|---|
| Spawns, invisible | RVV Reference Actor not set on the RVM; `SK_RailVehicle` name; mesh scale (a 6 cm loco is a units problem, Part 2 §10). |
| Cannot board | No CameraTransition / seat; seat's environment name; UCX collision missing so the player falls through. |
| HUD shows no levers | VHID InteractionEnvironmentComponent does not match the environment you occupy; input identifiers empty. |
| Levers move, loco does not | RVM graph writes to wrong node/parameter names; reverser link to motor polarity; brake default 1 with no release. |
| Moves, wheels static | ABP binding node/parameter; Axles list; Wheels_FullRotation not assigned to the evaluator. |
| Door clip plays the lever | VirtualHIDSequenceEntries row points at the wrong sequence or the view hitbox name instead of the RVM component. |
| Mirrored left/right | FBX exported with non-default axes; re-export with the defaults (Part 2 §10). |
| Will not couple to Dovetail stock | coupler_id mismatch; read it off a running vehicle (§6); ConnectivityData not recalculated. |
| Sounds vanish when you come back | No AudioToViewBindings; the RVV respawned and the audio stayed attached to the old one. |
| Floats above or sinks into the rails | `PhysicsBodyOffset` Z = half the height; regenerate RVD bounds. **community** |
| Jumps and twists when hauled | Two bogie nodes in the SimuGraph with the same offset sign. **community** |
| Imports a hundred times too big | Unapplied transforms / unit scale; Part 2 §10. **community** |
| Not found by the game's region filter | Custom region string; pick an existing region in the RVD. **community** |
| Packaged, but not in the spawn list in the game | No formation asset; place it with the community God Mode tool until one exists. **community** |

# 8 · Shipping the plugin

The inspection did not cover distribution; the community stream did, end to end, on TSW 5. Dovetail's FAQ sets the rules (no redistribution of Dovetail assets, no modified Dovetail trains); the mechanics below are the streamer's and worked for him. Before you plan a store product around TSW, read the current FAQ and the community wiki's publishing page and confirm what the licence permits commercially. **check**




**Figure 6.2 · The community packaging route on TSW 5.** Six steps from a working plugin to a package another player can drop into their game. community

1. **Content manifest.** In the plugin's root folder: right-click ▸ Miscellaneous ▸ Data Asset ▸ `ContentManifest`, named `ContentManifest`. Open it and add your RVD(s) under the defined rail vehicles. Without it the cook does not pick the vehicle up.
2. **Clean the game's DLC folder.** Close the editor. In the game install (`…/TS2Prototype/Content/DLC`) temporarily remove every third-party package, keeping only the DLC you bought; third-party packs present during the cook break it.
3. **Launch the editor through the community packer** (a small tool published on the Dovetail forum's PC Editor section, versions 2.1/2.2 at the time of the stream) from a command prompt. Launched normally, the cook and the package steps do not run.
4. **Cook.** Choose cook mode, select your plugin, run. Warnings about old packages and about Dovetail assets referenced by the test timetable can be ignored; remove your test timetable from the map first if you want a quiet log.
5. **Distribute plugin.** Pick your plugin, tick the original plugin and any DLC you referenced (none for a standalone wagon), Distribute ▸ OK. The output lands in the editor's folder as a package of a few tens of MB.
6. **Install and test.** Copy the package into the game's `Content/DLC`, start the game: the vehicle appears in the depot list (the streamer's wagon showed under locomotives). Without a formation it is not offered by the spawn menu; place it with the God Mode tool or add the formation and re-cook.

## The thumbnail

Dovetail's forum (PC Editor section) hosts a *preview scene* plugin: install it like any plugin, open its map, add your formation to its timetable, rebuild, press Play and take the screenshot; crop it, import it into `Data/RVD` and set it as the RVD's `ThumbnailTexture`. That gives the standard TSW menu picture in two minutes. community

> **Keep the master files yours**
>
> As with the Steam-partner case in the store guide: whatever the distribution route, only the cooked plugin leaves your machine. The .blend, the textures and the SimuGraph source stay with you.

# 9 · What a finished vehicle contains

Thomas's plugin: 317 assets. Only seven are structural (RVD, RVM, RVV, RVA, SA, SK_Ext, ABP_Ext); everything else is polish. That is the whole argument for building the boilerplate first. verified

| Type | Count | Type | Count | Type | Count |
|---|---|---|---|---|---|
| SoundWave | 114 | SkeletalMesh | 14 | SoundAttenuation | 6 |
| Texture2D | 56 | Material / Instance | 8 / 20 | PhysicsAsset | 5 |
| AnimSequence | 28 | AnimBlueprint | 8 | Blueprints (RVM, RVV, RVA, preview, character) | 5 |
| SoundCue | 15 | StaticMesh | 8 | Skeleton | 3 |
| CurveFloat | 8 | Character hair / outfit / collection | 6 / 4 / 1 | RVD, SA, DataTables, RewardLevel, SoundMix, LODSettings, WidgetBP | 1 each |